

DWH Schema Modelling Guide

Star • Snowflake • Galaxy (Fact Constellation)

Star Schema

Snowflake Schema

Galaxy Schema

What You'll Learn:

- Core concepts and architecture of each schema type
- Detailed examples with real-world retail / sales use cases
- Key differences: normalization, performance, complexity
- When to use Star vs Snowflake vs Galaxy schema
- Best practices for modern Data Warehouse design

Shashank Mishra | Staff Data Engineer & Data Architect

YouTube: @shashank_mishra · Instagram: @_shashank_219 · LinkedIn: @shashank219

www.growdataskills.com

01 | Introduction to Data Warehouse Modelling

What is Data Warehouse Modelling?

DWH modelling defines how data is organized and accessed in an analytical system. Schema choices directly impact query speed, storage efficiency, ETL complexity, and maintainability.

Why Schema Design Matters

Query Performance

Optimized schema = faster analytical queries with fewer JOIN operations.

Storage Efficiency

Normalized schemas reduce data redundancy and lower overall storage needs.

Scalability

Well-designed schemas scale with data volume and evolving business needs.

OLTP vs OLAP – Key Distinction

Attribute	OLTP (Transactional)	OLAP / DWH (Analytical)
Purpose	Day-to-day operations	Analytical reporting
Schema Design	Highly Normalized (3NF)	Denormalized (Star/Snow)
Query Type	Short, frequent writes	Complex, long reads
Data Volume	GB range	TB – PB range
Users	App users, developers	Analysts, data scientists

Three Schema Patterns at a Glance

Star Schema

1 Fact + N Dims
Denormalized
Simplest & Fastest

Snowflake Schema

1 Fact + N Dims
+ Sub-Dims (3NF)
Normalized

Galaxy Schema

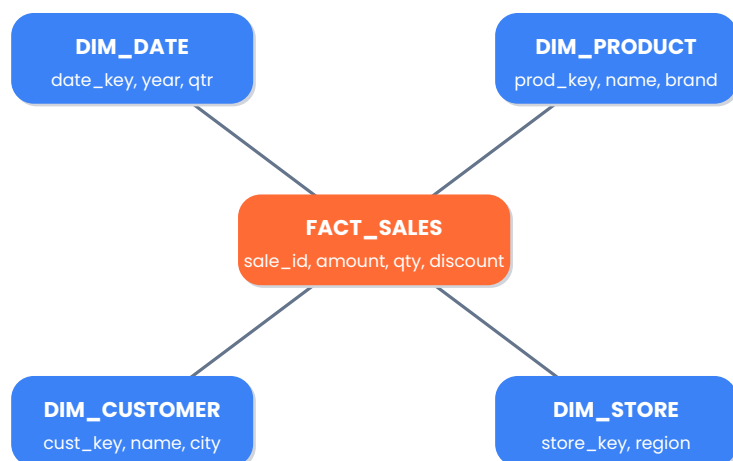
N Facts + N Dims
Shared Dimensions
Enterprise Scale

Complexity Increases

02 | Star Schema

Definition

A Star Schema has one central Fact Table surrounded by flat, denormalized Dimension Tables. The shape resembles a star. Dimension tables are NOT broken into sub-tables.



Key Characteristics

- Single central Fact Table
- Multiple flat Dimension Tables
- Dimensions are denormalized
- Minimal JOIN operations
- Fastest query performance
- Redundant data is acceptable
- Best for BI & dashboards

Example Schema — Retail Sales

FACT_SALES (Fact Table)		
sale_id (PK)	BIGINT	Surrogate key
date_key (FK)	INT	DIM_DATE
product_key (FK)	INT	DIM_PRODUCT
customer_key (FK)	INT	DIM_CUSTOMER
store_key (FK)	INT	DIM_STORE
sales_amount	DECIMAL	Revenue
quantity_sold	INT	Units sold

DIM_PRODUCT (Denormalized Dim)		
product_key (PK)	INT	Surrogate key
product_id	VARCHAR	Natural/source key
product_name	VARCHAR	Full name
category_name	VARCHAR	Flat — denormalized
sub_category	VARCHAR	Flat — denormalized
brand_name	VARCHAR	Flat — denormalized
unit_price	DECIMAL	Current price

Advantages

- Simpler queries — fewer JOINS
- Fastest read performance
- Easy for BI tools to navigate
- Simple to build and maintain

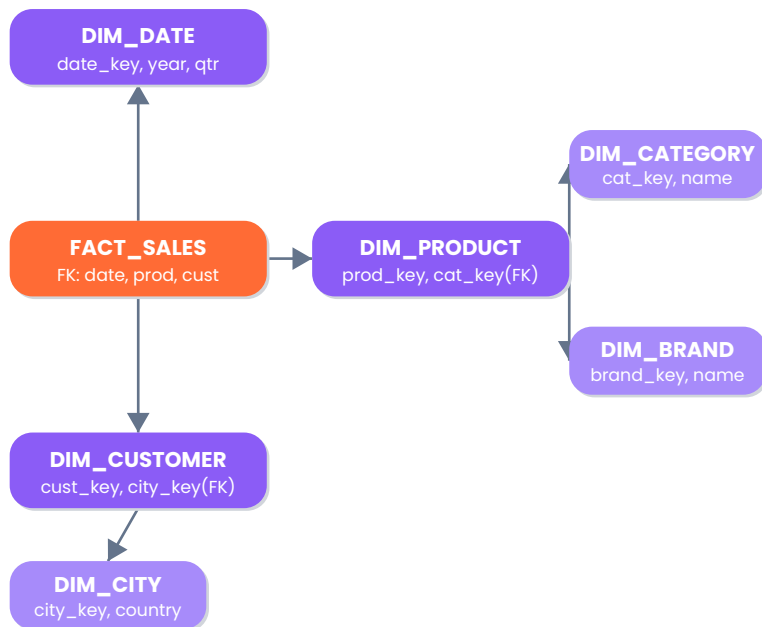
Disadvantages

- Data redundancy in dim tables
- Higher storage consumption
- SCD updates can be complex
- Not ideal for very large dims

03 | Snowflake Schema

Definition

Snowflake Schema extends the Star Schema by normalizing dimension tables into multiple related sub-tables (3NF). The shape resembles a snowflake due to branching dimension tables.



Key Characteristics

- Same central Fact Table
- L1 dimensions connect to fact
- L2 sub-dims break down L1
- All dims normalized to 3NF
- More tables more JOINS
- Reduces data redundancy
- Better for large-scale DWH

Star vs Snowflake — Normalization Comparison

DIM_PRODUCT (Star — Denormalized)	
product_key	101
product_name	iPhone 15
category_name	Electronics
sub_category	Smartphones
brand_name	Apple
brand_country	USA

Snowflake — 3 Normalized Tables		
DIM_PRODUCT	DIM_CATEGORY	DIM_BRAND
product_key	cat_key	brand_key
product_name	category	brand_name
cat_key(FK)	sub_category	country
brand_key(FK)		

Advantages

- Eliminates data redundancy
- Lower storage footprint
- Easier dimension attribute updates
- Better data integrity (3NF)

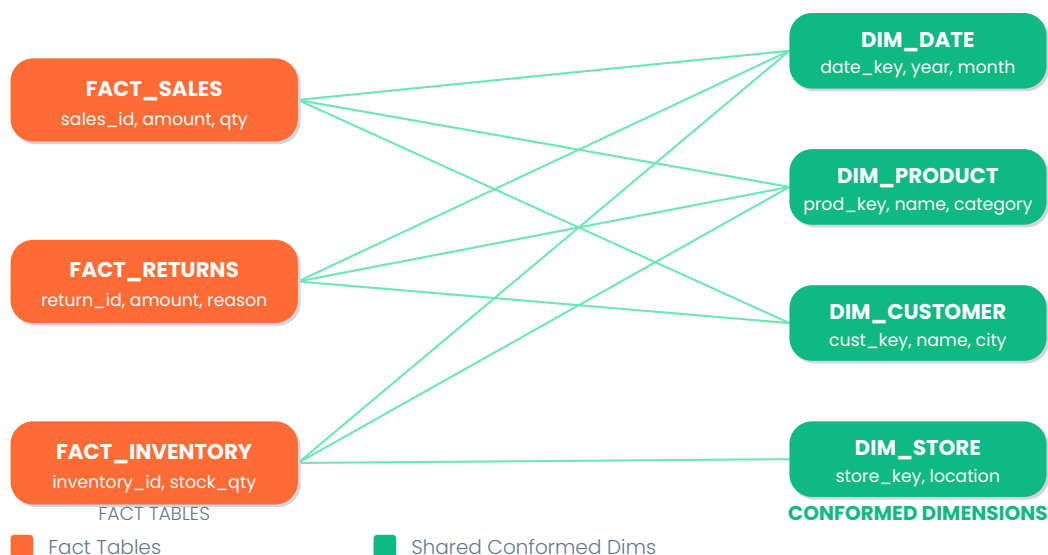
Disadvantages

- Complex queries — many JOINS
- Slower query execution
- Harder for BI tools to navigate
- More complex ETL pipelines

04 | Galaxy Schema (Fact Constellation)

Definition

Galaxy Schema (Fact Constellation) contains multiple Fact Tables that share common Dimension Tables. Shared dims are called Conformed Dimensions — consistent across all facts.



Key Characteristics

Multiple Fact Tables

Two or more fact tables capture different business processes (Sales, Returns, Inventory, Orders).

Conformed Dimensions

Shared dimension tables used across multiple facts — DIM_DATE, DIM_PRODUCT reused everywhere.

Enterprise Scope

Designed for large enterprises with complex analytical needs spanning multiple business domains.

Fact Constellation

The visual pattern of multiple facts sharing dims resembles a galaxy or constellation of stars.

Advantages

- Handles multiple business processes
- Reusable conformed dimensions
- Comprehensive enterprise analytics
- Supports cross-process analysis

Disadvantages

- Most complex to design & govern
- Very complex ETL pipelines
- Needs semantic layer for querying
- High upfront design investment

05 | Detailed Comparison: Star vs Snowflake vs Galaxy

Head-to-Head Comparison

Dimension	Star	Snowflake	Galaxy
Normalization	Denormalized	Normalized (3NF)	Mixed
Fact Tables	1	1	2+ (multiple)
Dim Structure	Flat tables	Hierarchical (nested)	Shared/conformed
No. of Tables	Few	Many	Many
JOINS Required	Minimal	Many	Complex multi-fact
Query Performance	Fastest	Medium	Varies by design
Storage Usage	Higher (redundancy)	Lower	Medium
ETL Complexity	Moderate	High	Very High
Data Redundancy	High	Low	Low (shared dims)
BI Tool Friendly			(needs layer)
Maintenance Effort	Low	Medium	High
Recommended Scale	Small-Medium DWH	Medium-Large DWH	Enterprise DWH
Learning Curve	Low	Medium	High

Quick Decision Guide — When to Choose Each Schema

Choose STAR Schema when...

Need fast, simple analytical queries
 Primary use: BI dashboards & reports
 Dimension cardinality is low-medium
 Team is new to DWH design

Choose SNOWFLAKE when...

Storage optimization is critical
 High-cardinality dimension attributes
 Data integrity is a top priority
 Dimension updates are frequent

Choose GALAXY when...

Multiple business processes to model
 Cross-domain analytics are required
 Need conformed / shared dimensions
 Building enterprise-scale DWH

06 | Real-World Examples & SCD Types

Industry Use Cases

E-Commerce / Retail

Star Schema

FACT_SALES DIM_CUSTOMER, DIM_PRODUCT, DIM_DATE, DIM_STORE

Fast dashboards for sales KPIs, product performance, regional revenue.

Banking & Finance

Snowflake Schema

DIM_ACCOUNT DIM_ACCOUNT_TYPE DIM_BRANCH DIM_REGION

Normalized dims for regulatory compliance and account hierarchies.

Healthcare / Insurance

Galaxy Schema

FACT_CLAIMS + FACT_ENCOUNTERS sharing DIM_PATIENT, DIM_DATE, DIM_DOCTOR

Cross-domain analytics: patient journey, claim patterns, outcomes.

Telecommunications

Galaxy Schema

FACT_CALLS + FACT_DATA_USAGE + FACT_BILLING sharing DIM_CUSTOMER

Unified analytics across calls, data usage, and billing cycles.

Slowly Changing Dimensions (SCD) – Overview

SCD defines how dimension data changes over time in a DWH. Applicable to all schema types — especially Star and Snowflake. The right SCD type ensures historical accuracy and clean analytical results.

SCD Type 0

Retain Original

Keep original value forever.
No updates, no history.
Used for static reference data.

SCD Type 1

Overwrite

Overwrite old value with new.
No history is maintained.
Simple — loses historical state.

SCD Type 2

Add New Row Most Common

Add a new row with eff. dates.
Full history is maintained.
Most widely used in production.

SCD Type 3

Add New Column

Add 'previous_value' column.
Limited history (1 version back).
Used for minor attribute changes.

Schema Design in the Modern Data Stack

Modern DWH platforms (Snowflake, BigQuery, Databricks, Redshift) support all three schema types natively. With columnar storage & modern query optimizers, performance gaps between Star & Snowflake have narrowed. dbt (data build tool) is the standard for schema transformation — staging dims facts pattern maps directly.

07 | Best Practices & Summary Cheatsheet

Best Practices for DWH Schema Design

Start with Star Schema

Begin simple. Only move to Snowflake when storage or cardinality demands it.

Design Conformed Dimensions Early

Plan shared dims upfront for Galaxy schemas to ensure cross-process consistency.

Plan SCD Types at Design Time

Decide SCD type per dimension early — SCD Type 2 is the most common in production.

Use Surrogate Keys

Always use integer surrogate keys for fact/dim joins — never natural business keys.

Follow Kimball Methodology

Ralph Kimball's bus matrix & dimensional modelling is the gold standard for DWH design.

Use dbt for Transformations

Use dbt models with clear staging, dim, and fact layers. Version-control your schema.

Final Cheatsheet — Star vs Snowflake vs Galaxy

Schema	Fact Tables	Dim Design	Query Speed	Best Use Case	Complexity
Star	1	Denormalized	Fastest	BI / SMB DWH	Low
Snowflake	1	Normalized 3NF	Medium	Large DWH	Medium
Galaxy	Multiple	Shared/Conform	Varies	Enterprise DWH	High

Want to Become a Top Data & AI Professional in 2026?

Follow Shashank Mishra for daily Data & AI content:

YouTube: @shashank_mishra · Instagram: @_shashank_219 · LinkedIn: @shashank219

Visit www.growdataskills.com